16/10/2026
Codebasics

Object Detection using YOLO v4 and Pretrained Model

Joseph Redmon
- 2016 — YOLO v1
- 2017 — YOLO v2 (67 FPS) 67 Frame Per Second
- 2018 — YOLO v3

2020 → YOLO v4 (by alexey AB) github repo darknet

Lec29
NPTEL
IIT of Sunny Bgalore

Object Detection

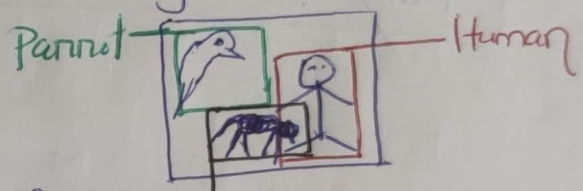
Classification, Localization, and Object Detection



→ Single Class Label for the whole Image



Box in the Image (x, y, w, h)
- Coordinate of the Center of Bounding Box
- (width, height) of Bounding Box



Classify each Object to a Specific Class & Bounding Box for the Object.

[Object Detection = Classification + Localization]

→ Localization as a Regression Problem



→ Neural Network → (x, y, w, h)
4 Predicted Values (Bounding Boxes Coordinates)

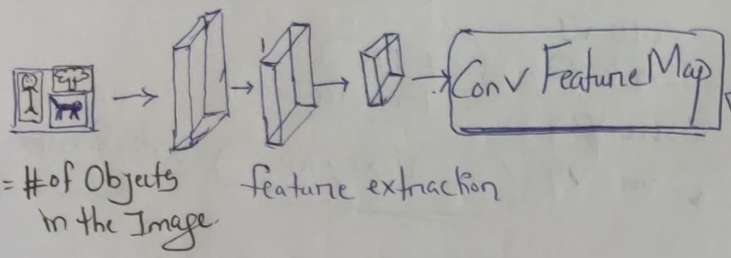
Ground Truth → Connected Bounding Box Coordinates.

Error → L2 distance between the Predicted Values & the Ground ~~Truth~~ Truth.

↓

$\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$ (Least Squared Loss (L2))

→ Can Classification and Localization be done together? Yes



Q) Can CNN's trained for Classification be used for Object detection?

Yes, using Sliding Window.



Multiple Positions to be Scanned which is Computationally expensive.

→ Selective Search Algorithm: (for Object Detection)

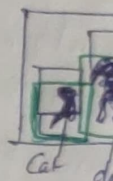
① Starts with Over Segmentation of the Image called Super pixels. (Similar neighbouring regions Merged repeatedly to create a Larger Regions.)

→ (colour, Texture, Size Similarity, Shape Compatibility)

helps to region proposal generation

② Bounding Boxes around the region. is Recorded. called as a Region proposal.

→ Region Prop



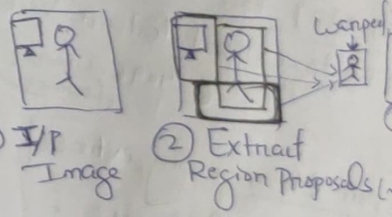
FP ☐ BlueBoxes

TP ☐ greenBoxe

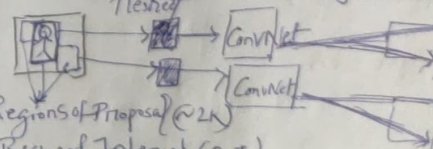
R-CNN

— This R-CNN fram

RCNN → Regions with



- First Proposes Candidate R
- Feed these regions throu
- Classify each region us



→ After extracting features for eac are trained separately for ea A separate Linear Regressor is t the bounding box coordinates.

→ Region Proposals - Based on Selective Search Algorithm

(Class Scores)
(K numbers)



① finds Regions that are Likely to contain an Object.

② Class agnostic Object detection
Irrespective of which class it is!

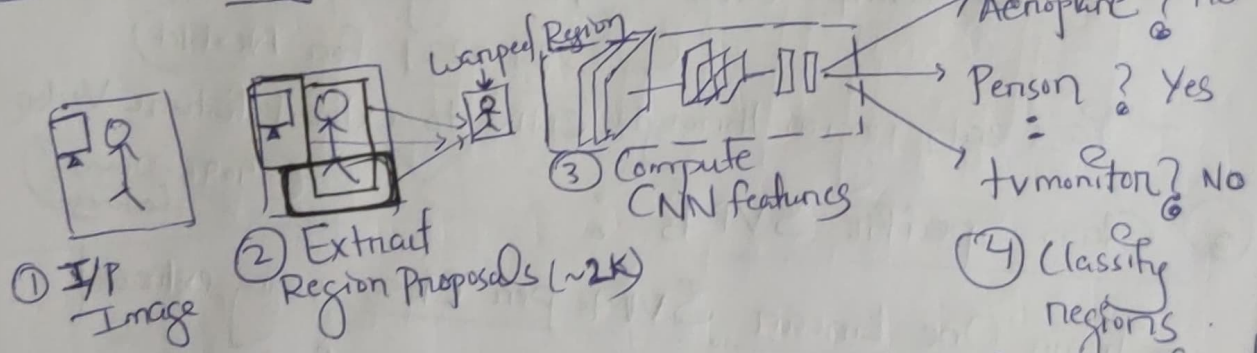
FP ☐ BlueBoxes - FalsePositive

TP ☐ greenBoxes - TruePositive

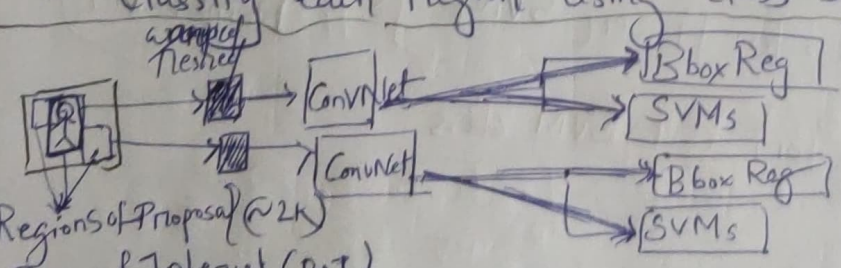
R-CNN Framework

- This R-CNN framework works on Region Proposals

RCNN → (Regions with CNN features)



- First Proposes Candidate Regions in an Image (Selective Search Algorithm) → high chances Region of getting an Object
- Feed these regions through a CNN to extract Rich features.
- Classify each region using class Specific Linear SVMs.



→ After extracting features for each Region of Interest, SVMs are trained separately for each class.

A separate Linear Regressor is trained independently to refine the bounding box coordinates.

R-CNN Training

① Region Proposal Generation :

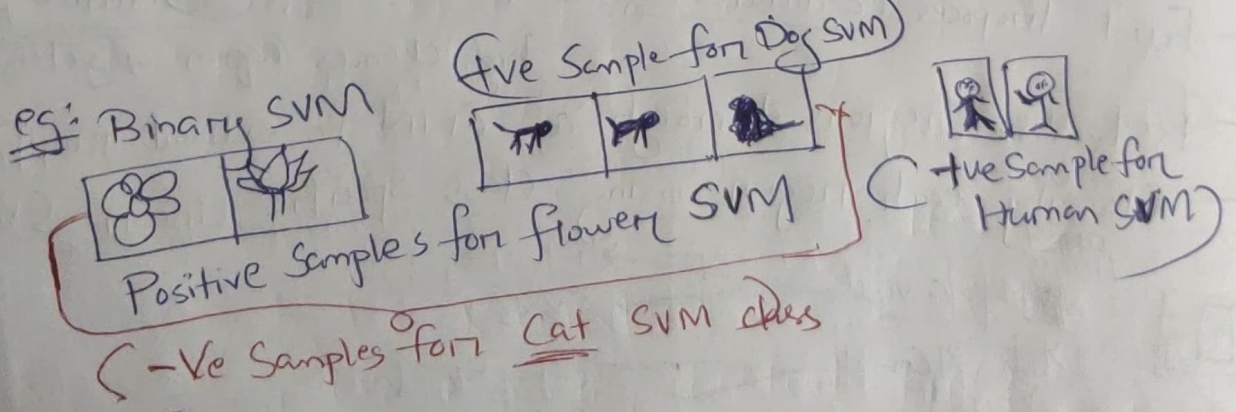
- Category-independent Selective Search.
- Provides candidate bounding boxes for Objects.
- Produce a Set of candidate Regions, such that almost every true object in the Image overlaps reasonably well with at least one candidate box.

② Feature extraction with CNN :

- Download classification model AlexNet (1000 class)
- Finetune it for Pascal VOC 2007 dataset (20 Object Class)
- Crop each region from the Original Image
- Resize/warp each cropped image to fixed size. (227 x 227 for AlexNet)
- Pass each region through CNN to extract feature Vector (4096-Dimensional features)

③ Class Specific SVMs :

- Train One Linear SVM per Class using extracted features.



Training with SVMs



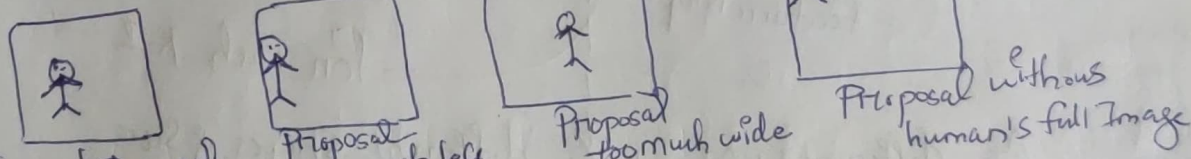
→ Two Classes of Objects
  → Human figure
  → Horse

↓

→ There will be Two SVMs
  ① ⇒ Horse Class — This SVM (Binary Class Horse or Not ? (+ve) (-ve))
  ② ⇒ Human Class . (+ve / -ve)

→ Feed the features extracted from CNN. to All SVMs
  - Each SVM outputs a Score
  - The Class with highest Positive Score wins.
  - If all SVM scores negative — Background

⇒ Bounding Box Regression (



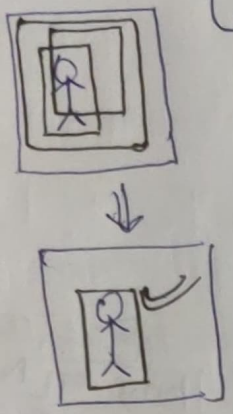
→ Regressor Predicts the Offsets (dx, dy, dw, dh) between the Proposed box and the ground truth box. This is used to Compute refined box.

→ The bounding box regression helps improve the accuracy of Object localization by adjusting the initially Proposed bounding box to better fit the Objects actual boundaries.

→ Non Maximum Suppression

↳ removes duplicate detections by retaining only regions with highest IoU (statistic) score for each class.
↳ increases accuracy.

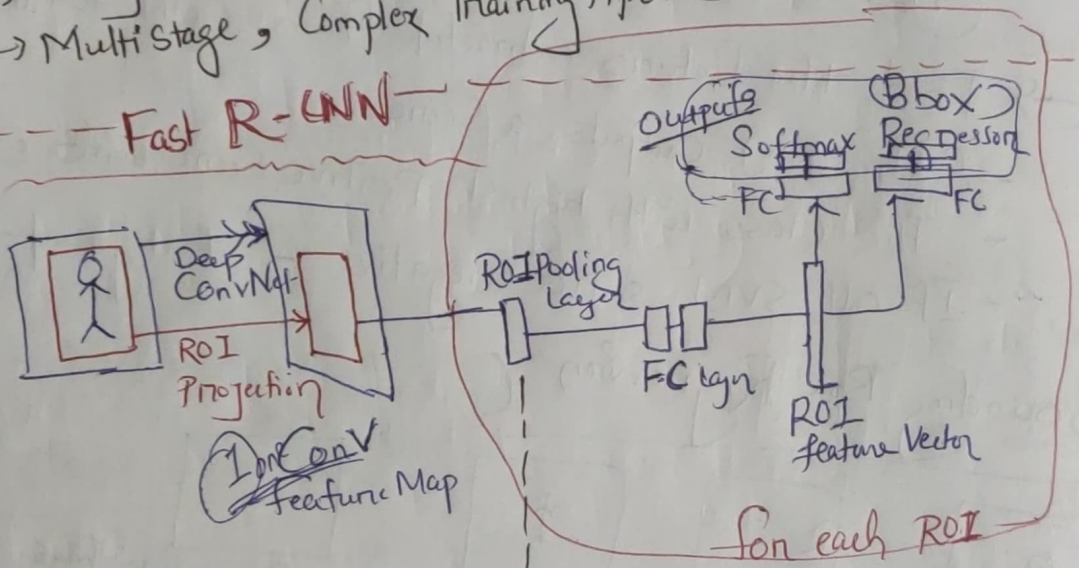
Intersection Over Union



Limitations of R-CNN:

↳ Very Slow at Test Time
↳ Multi stage, Complex Training Pipeline.

Fast R-CNN



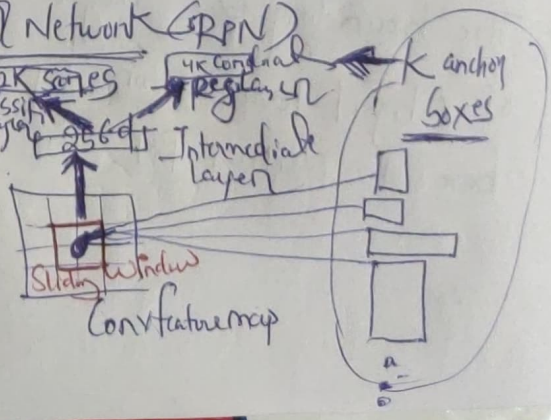
As different ROIs have different Shapes & Sizes.
ROI pooling Standardize them So they can be Processed by Fully Connected Layers.

Eg Proposal size (h x w) → fixed Size Output after ROI (H x W)
eg (7x7)

Faster-R-CNN

↳ it Uses Region Proposal Network (RPN)

- Small NN that Slides over the Conv feature map of the entire Image.
- Automatically Predicts (object possibility)
  ↳ Objectness Score
  ↳ Bounding box coordinates
- RPN replaces Selective Search.



	RCNN	Fast RCNN	Faster RCNN
Test time per Image (with Proposals)	50secs	2secs	0.2secs

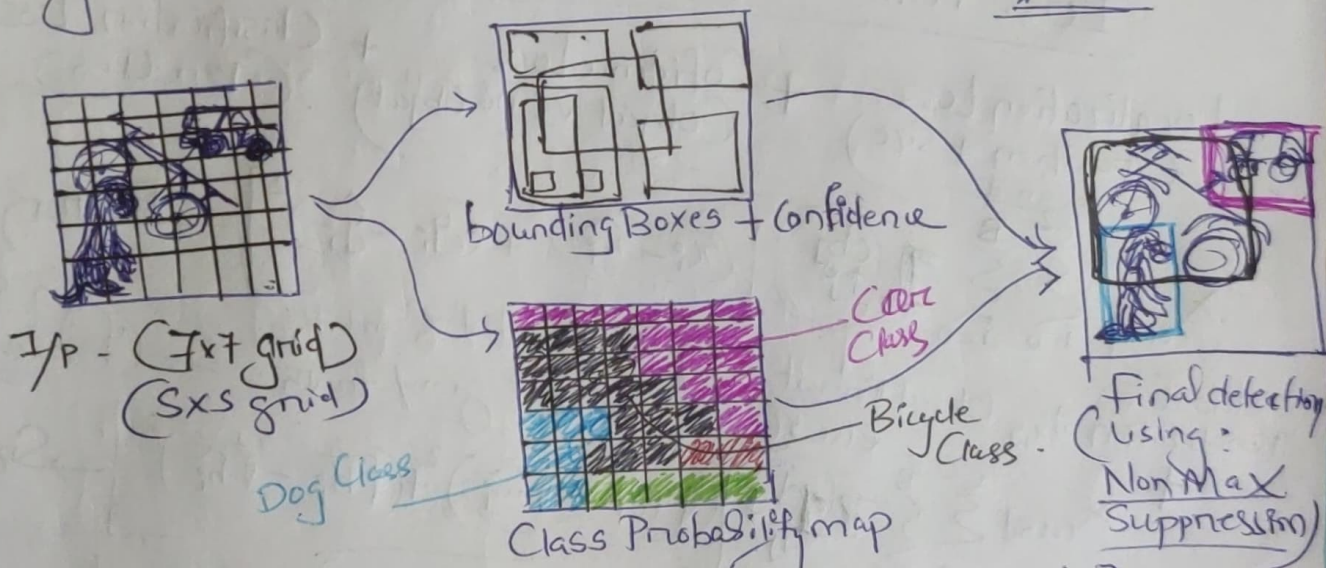
Faster R-CNN

- Extremely fast (region Proposal is computed ≈ 10ms on GPU)
- Fully trainable end-to-end
- More accurate (Learned Proposals are better aligned with the detector)

17/10/2020 →

You Look Only Once (YOLO-v1) for Object detection

Predicts bounding boxes & Class Probability all at Once.
using end-to-end neural network. (You Only Look Once) YOLO



- Each grid cell – Predicts Multiple Bounding Boxes (x, y, w, h)
  & Also with Confidence Score.
  - Probability(Object exist) x IOU truth pred
- Class Probability map: each grid predicts conditional Probability
  $Pr(Class_i / Object) = P(Dog\ Class / Object\ exists)$
  - One Set of Probabilities Per grid cell → dog 0.90, bicycle 0.06, can 0.04
  - $Pr(Class_i/Object) * Pr(Object) * IOU^{truth}_{pred} \equiv Pr(Class_i) * IOU^{truth}_{pred}$ → Confidence Score for the bounding box
- Class Specific Confidence Score for Each box.

⇒ Non Max Supression: Keeps only boxes having highest Probability detection score.

Architecture of YOLO v1 Model

24 Convolutional Layers followed by 2 Fully Connected Layers.

- 24 Convolutional Layers → feature extraction
- 2 Fully Connected Layers → Predicts probabilities & Coordinates.

YOLOv1 treats detection as a Regression Problem.

For each grid cell, it Predicts

- Bounding Box Coordinates (x, y, w, h)
- Confidence Score (C) for Object
- Class Probabilities $(p(\text{class}_i))$

Loss function in YOLO v1

Localization Loss (position + size) + Confidence Loss (Object vs no Object) + Classification Loss (per class)

$$\lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \rightarrow \text{Position}$$

$\hat{x}_i$: predicted x coordinate; x_i: actual ground truth.

λ_{coord}: default 5, penalization of x, y.

$\mathbb{1}_{ij}^{obj}$: Object Present → 1, else 0.

$\sqrt{}$ → Relative w-loss: 200 − 190 = 10, 20 − 10 = 10

$$+ \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \rightarrow \text{Size}$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 \rightarrow \text{Confidence Loss}$$

ground truth confidence = 1

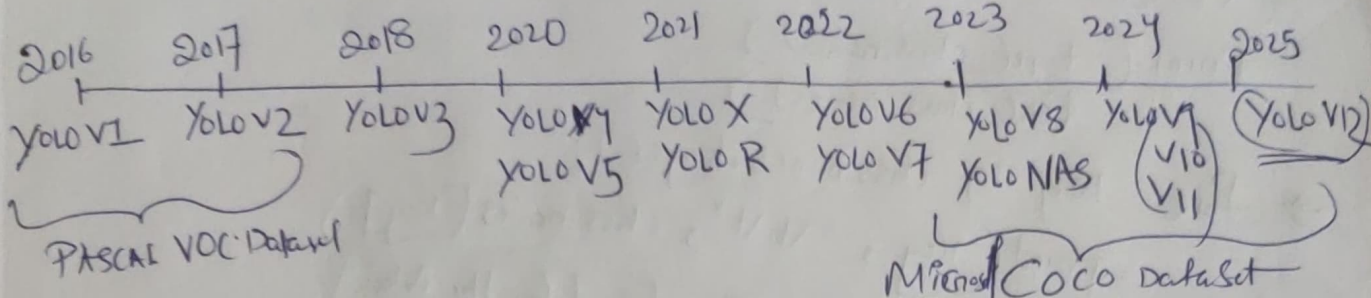
$$+ \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^{B^2} \mathbb{1}_{ij}^{noObj} (C_i - \hat{C}_i)^2$$

λ_{noobj}: default 0.5; $\mathbb{1}_{ij}^{noObj}$: 1 if no object; ground truth confidence → 0

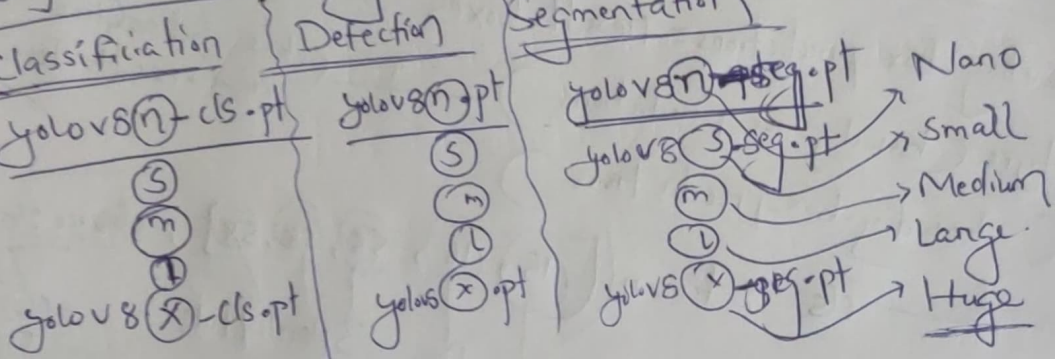
$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{obj} \sum_{c \in \text{Classes}} (P_i(c) - \hat{P}_i(c))^2 \rightarrow \text{Classification Loss}$$

→ Penalizes Incorrect Class Predictions

Variants of YOLO :-



→ Object Detection Using YOLO (Pretrained Mode)



→ !pip install ultralytics -

→ Yolo is trained on coco dataset (80 objects)

- from ultralytics import YOLO
- model = YOLO("yolov8m.pt") #medium Sized.
  model ——— model architecture.
- type(model)

⇒ test_image_path = "————————"
test_image = cv2.imread(test_image_path)
print("testimage.shape",)
print("Sample test image. \n")
plt.imshow(test_image)

→ #Now Detect the Object In the Image
results = model.predict(test_image)
print("\n type(results) : \n", type(results), "\n\n")
List

- print("\n Object detection Results : \n", results)
  len(results) ——— (1) images as Input for testing
  result = results[0]
  result [ultralytics.engine.results.Results object with attributes
  boxes : , masks, names, obb : , orig_img : ,
  keypoints, orig_shape : , path, probs, speed : {}

→ class_names = result.names

Print(type(class_names))
len(class_names))

→ for key in class_names.keys():
print(key, "\t \t", class_names[key])

#Analyze bounding boxes returned by the Model

bounding_box_predictions = result.boxes

Print(type(bounding_box_predictions)) → <class 'ultralytics.engine.results.Boxes'>

Print(bounding_box_predictions)

cls : tensor([15, 16, 58, 58, 58], device='cuda:0')
15 → Cat
16 → Dog
58 → 3 planted Pots.
cuda → NVIDIA GPU

conf : tensor([0.95, 0.91, 0.52, 0.48, 0.39]),..

data : tensor([[... , 0.94, 15]])
(x1 coordinate, y1 coordinate, x2, y2, Confidence, class)

id : None
is_track : False
orig_shape : (765, 1470)
shape : torch.Size([5, 6])
5 → 5 object (row)
6 → 6 value (column)

xyxy : tensor([[389.3092, 383, 593, 471],
[ ],
[ ],
[ ],
[ ]])
5 objects

Confidence

Normalized → xyxyn :

(xmin, ymin, xmax, bottom right) → xyxy :

xywhn :

xywh :

#Visualize Bounding Boxes

from ... .colors.patches import cv2_imshow

bounding_box_xyxy_coordinates = bounding_box_predictions.xyxy

type( ) → tensor

bounding_box_xyxy_coordinates_list = bounding_box_xyxy_coordinates.tolist()

Print("Number of Boxes detected : ", len(bounding_box_xyxy_coordinates_list))

→ for box_index in range(len(bounding_box_xyxy_coordinates_list)):
print("Showing box :", box_index+1, "\n")
x1, y1, x2, y2 = bounding_box_xyxy_coordinates_list[box_index]
x1 = int(x1)
x2 = int(x2)
y2 = int(y2)
y1 = int(y2)
cv2.rectangle(original_test_image, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2_imshow(original_test_image)
print("\n\n")

Bounding box Color (Green)
RGB
(0, 255, 0)
thickness.

* OpenCV (BGR) img = cv2.imread("image.JPG")
cv2.imshow(image)

Matplotlib (RGB) img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)

PIL (RGB)
↳ from PIL import Image
img = Image.open('image.JPG')
img.show()